

Three forms of co-simulation interfaces

Memory-mapped interface. In this format, there is a set of addresses in the address sector of the core, which are accessed by both hardware and software running on the core. Both access the same addresses by means of multiplexing and time-sharing. Advantage of this interface is that it can be ported on to any system, with minimal changes.

Specialised co-processor I/O interface. A specialised co-processor defines an I/O port into the pipeline of the processor, and thus allows for implementation of application-specific instruction set processor concepts. The special function unit interfaces are triggered by special, reserved instructions. This type of interface is implemented using armfsu2x2, armfsu2x1 and armfsu3x1.

Fast-Simplex link interface. This defines a dedicated co-processor port on ARM, which is emulated using memory-reads and memory-writes in ARM software. Like the previous form, this interface also carries a higher bandwidth than the memory-mapped alternative. It is implemented using armfslslave and armfslmaster.

the I/O mechanism provided by the core. Command line of gplatform is:

```
gplatform [-d] -c max_cycles gezel_file
```

GEZEL code-generation tool.

fdlhvd is the tiny spark that sets it apart. It allows you to convert any code that you can write, and which simulates correctly, to VHDL code. GEZEL engineers have used Sourceforge to give detailed examples for

using fdlhvd, in case of starting up troubles. Command line of fdlhvd is:

```
fdlhvd [-d] [-i] [-s] [-c clock reset]
[<filename>]
```

Is it fast enough

Patrick Schaumont writes at *rijndael.ece.vt.edu*, "In general, speed of a good instruction-set simulator is far higher than that of GEZEL kernel."

He explains that this is because an instruction-set simulator is developed with the architecture of the processor it must model in mind, which is not possible for GEZEL kernel. Also, GEZEL kernel uses scripted simulation rather than compiled simulation.

For optimisation, GEZEL simulator uses a strategy of sleep or awake modes, which is important for co-simulation. "If this is possible, you should develop GEZEL hardware model in such a way that periods of idle or inactive operation also imply no datapath register changes and no state changes in GEZEL controllers," adds Schaumont. This will enable GEZEL simulator to enter sleep mode, while only the instruction-set simulator keeps on running. The resulting simulation speed will improve greatly. **EFY**

Ashwin Gopinath, an engineer, is a freelance writer for EFY



Would You Like More DIY Circuits?

We Have Thousands!

VISIT TODAY

electronicsforu.com

If it's electronics, it's here